

2025년 3월 29일 토요일

Week4_예습과제_백재은

Ch.4-5 ~ 4-11

4-5 GBM(Gradient Boosting Machine)

1) GBM의 개요

부스팅 알고리즘: 여러 개의 약한 학습기를 순차적으로 학습-예측하면서 잘못 예측한 데이터에 가중치를 부여하여 오류를 개선해 나가는 학습 방식

Ex. AdaBoost, Gradient Boost

AdaBoost: 오류 데이터에 가중치를 부여하면서 부스팅을 수행하는 알고리즘

GBM: 경사 하강법을 이용하여 가중치 업데이트

경사 하강법: 오류식을 최소화하는 방향성을 가지고 반복적으로 가중치 값을 업데이트하는 것

2) GBM 하이퍼 파라미터

loss: 비용함수 지정

learning_rate: 학습 진행 시 적용하는 학습률

n_estimator: weak learner의 개수

Subsample: 데이터 샘플링의 비율

4-6 XGBoost(eXtra Gradient Boost)

1) XGBoost 개요

- 느린 수행 시간, 과적합 규제 부재 해결
- 병렬 CPU 환경에서 병렬 학습 가능
- 분류에 있어 타 머신러닝보다 좋은 성능
- Tree pruning
- 자체 내장된 교차 검증, 결손값 자체 처리
- 핵심 라이브러리: C / 파이썬 패키지 제공

2) XGBoost 하이퍼 파라미터

- 일반 파라미터: 일반적인 실행 시 스레드 개수, 모드 선택을 위한 파라미터 / `booster(gbtree, gbliner)`, `silent`, `thread`
- 부스터 파라미터: 트리 최적화, 부스팅, `regularization` 관련 파라미터 / `eta`, `num_boost_rounds`, `min_child_weight`[default = 1], `gamma`, `max_depth`, `sub_sample`, `colsample_bytree`, `lambda`, `alpha`, `scale_pos_weight`
- 학습 태스크 파라미터: 학습 수행 시의 객체 함수, 평가 지표 설정 파라미터 / `objective`, `binary:logistic`, `multi:softmax`, `eval_metric`
- 과적합 문제 발생 시: `eta` 값 하향 조정, `max_depth` 값 하향, `min_child_weight` 값 상향, `gamma` 값 상향, `subsample`, `colsample_bytree` 조정
- 조기 중단: 부스팅 반복 횟수에 도달하지 않더라도 더 이상 예측 오류가 개선되지 않으면 반복을 끝까지 수행하지 않고 중지

4-7 LightGBM

1) 개요

- 장점: 학습 시간이 짧음, 메모리 사용량 적음, 카테고리형 피처의 자동 변환과 최적 분할
- 단점: 과적합 발생 쉬움
- 지프 중심 트리 분할 방식: 최대 손실값을 가지는 리프 노드 지속 분할을 통해 규칙 트리 생성, 예측 오류 손실 최소화

2) 하이퍼 파라미터

주요 파라미터

- `num_iteration`: 반복 수행하려는 트리의 개수 지정
- `learning_rate`: 0과 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값
- `max_depth`: 트리 기반 알고리즘의 `max_depth`와 일치
- `min_data_in_leaf`: 결정 트리의 `min_samples_leaf`과 일치
- `num_leaves`: 하나의 트리가 가질 수 있는 최대 리프 개수
- `boosting`: 부스팅의 트리 생성 알고리즘 기술 (`gbdt`, `rf`)
- `bagging_fraction`: 트리가 커져서 과적합되는 것을 제어하기 위해 데이터 샘플링 비율 지정

- feature_fraction: 개별 트리를 학습할 때마다 무작위로 선택하는 피처의 비율
- lambda_l2: L2 regulation 제어를 위한 값
- lambda_l1: L1 regulation 제어를 위한 값

Learning Task 파라미터

- objective: 최솟값을 가져야 할 손실함수 정의

3) 하이퍼 파라미터 튜닝 방안

- 기본 튜닝: num_leaves의 개수를 중심으로 min_child_samples(min_data_in_leaf), max_depth를 함께 조정하면서 모델의 복잡도를 줄이는 것
- learning_rate를 작게 하면서 n_estimators를 크게 하는 것
- 과적합 제어: reg_lambda, reg_alpha와 같은 regularization 적용
- 학습 데이터 사용 피처 개수, 데이터 샘플링 레코드 개수를 줄이기 위해 colsample_bytree, subsample 파라미터 적용

4-8 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝

기존: XGBoost, LightGBM 적용 시 하이퍼 파라미터 개수 축소 및 개별 하이퍼 파라미터의 범위 축소 필요

1) 개요

베이지안 최적화: 목적 함수 식을 제대로 알 수 없는 블랙박스 형태의 함수에서 최대 또는 최소 함수 반환 값을 만드는 최적 입력값을 가능한 적은 시도를 통해 빠르고 효과적으로 찾아주는 방식 / 새로운 데이터를 입력받았을 때 최적 함수를 예측하는 사후 모델을 개선해 나가면서 최적 함수 모델 구축

베이지안 확률: 새로운 사건의 관측이나 새로운 샘플 데이터를 기반으로 사후 확률을 개선해 나가는 방식

구성 요소: 대체 모델, 획득 함수

대체 모델: 획득 함수로부터 최적 함수를 예측할 수 있는 입력값을 추천 받은 뒤 이를 기반으로 최적 함수 모델 개선

획득 함수: 개선된 대체 모델을 기반으로 최적 입력값 계산

4-10 분류 실습 - 캐글 신용카드 사기 검출

1) 언더 샘플링과 오버 샘플링의 이해

P. 레이블이 불균형한 분포를 가진 데이터 세트를 학습시킬 때 예측 성능의 문제 발생

-> 이상 레이블을 가지는 데이터 건수가 정상 레이블을 가진 데이터 건수에 비해 너무 적어 발생

-> 극도로 불균형한 레이블 값 분포로 인한 문제점 해결을 위해 적절한 학습 데이터를 확보하는 방안 필요 = 오버 샘플링, 언더 샘플링

언더 샘플링: 많은 데이터 세트를 적은 데이터 세트 수준으로 감소시키는 방식

오버 샘플링: 이상 데이터와 같이 적은 데이터 세트를 증식하여 학습을 위한 충분한 데이터를 확보하는 방법

2) 이상치 데이터 제거 후 모델 학습/예측/평가

- 이상치 데이터(Outlier)은 전체 데이터의 패턴에서 벗어난 이상 값을 가진 데이터
- IQR 방식: 사분위 값의 편차를 이용하는 기법, 박스 플롯 방식으로 시각화 가능

3) SMOTE 오버 샘플링 적용 후 모델 학습/예측/평가

4-11 스택킹 앙상블

스택킹: 개별 알고리즘의 예측 결과 데이터 세트를 최종적인 메타 데이터 세트로 만들어 별도의 ML 알고리즘으로 최종 학습을 수행, 테스트 데이터를 기반으로 다시 최종 예측을 수행하는 방식

메타 모델: 개별 모델의 예측된 데이터 세트를 다시 기반으로 하여 학습하고 예측하는 방식

핵심: 개별 모델의 예측 데이터를 각각 스택킹 형태로 결합해 최종 메타 모델의 학습용 피쳐 데이터 세트와 테스트용 피쳐 데이터 세트 만드는 것